

打印输出

2025年12月19日 10:04

```
In [1]: import pandas as pd
import numpy as np
```

```
In [2]: new_column_names = [
        'user_id',
        'item_id',
        'category_id',
        'behavior_type',
        'timestamp'
    ]
df = pd.read_csv('UserBehavior.csv', header=None, names=new_column_names)
```

```
In [3]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100150807 entries, 0 to 100150806
Data columns (total 5 columns):
#   Column          Dtype
---  ---
0   user_id         int64
1   item_id         int64
2   category_id     int64
3   behavior_type   object
4   timestamp       int64
dtypes: int64(4), object(1)
memory usage: 3.7+ GB
```

```
In [4]: df.head()
```

```
Out[4]:
```

	user_id	item_id	category_id	behavior_type	timestamp
0	1	2268318	2520377	pv	1511544070
1	1	2333346	2520771	pv	1511561733
2	1	2576651	149192	pv	1511572885
3	1	3830808	4181361	pv	1511593493
4	1	4365585	2520377	pv	1511596146

```
In [5]: df.isnull().sum()
```

```
Out[5]: user_id      0
item_id      0
category_id   0
behavior_type 0
timestamp    0
dtype: int64
```

```
In [6]: df.drop_duplicates(subset=['user_id', 'item_id', 'timestamp'], keep='first', inplace=True)
```

```
In [7]: df['datetimes'] = pd.to_datetime(df['timestamp'], unit='s')
```

```
In [8]: df['dates'] = df['datetimes'].dt.date
df['times'] = df['datetimes'].dt.time
df['hours'] = df['datetimes'].dt.hour
```

```
In [9]: df.head()
```

```
Out[9]:
```

	user_id	item_id	category_id	behavior_type	timestamp	datetimes	dates	
0	1	2268318	2520377	pv	1511544070	2017-11-24 17:21:10	2017-11-24	17
1	1	2333346	2520771	pv	1511561733	2017-11-24 22:15:33	2017-11-24	22
2	1	2576651	149192	pv	1511572885	2017-11-25 01:21:25	2017-11-25	01
3	1	3830808	4181361	pv	1511593493	2017-11-25 07:04:53	2017-11-25	07
4	1	4365585	2520377	pv	1511596146	2017-11-25 07:49:06	2017-11-25	07

```
In [10]: #时间范围
print(df['datetimes'].max())
print(df['datetimes'].min())

2037-04-09 05:22:35
1902-05-07 22:32:46

In [11]: start_date = '2017-11-25 00:00:00'
end_date = '2017-12-03 23:59:59'

df = df[(df['datetimes'] >= start_date) & (df['datetimes'] <= end_date)]

In [12]: len(df)

Out[12]: 98914480
```

获客异常

```
In [13]: #随机抽样100000条数据
temp_behavior = df.sample(n=100000)

In [14]: #筛选pv
pv_data=temp_behavior[temp_behavior['behavior_type']=='pv']

daily_stats = pv_data.groupby('dates').agg(
    pv=('user_id', 'count'),
    uv=('user_id', 'nunique')
)

# 计算人均访问量 (PV/UV)
daily_stats['puv'] = (daily_stats['pv'] / daily_stats['uv']).round(1)
```

In [15]: `daily_stats`

Out[15]:

	pv	uv	puv
dates			
2017-11-25	9405	9252	1.0
2017-11-26	9503	9329	1.0
2017-11-27	9043	8883	1.0
2017-11-28	9047	8896	1.0
2017-11-29	9377	9201	1.0
2017-11-30	9559	9405	1.0
2017-12-01	10161	9996	1.0
2017-12-02	12570	12389	1.0
2017-12-03	10881	10678	1.0

In [16]:

```
# 1. 过滤全量数据中的 PV 行为
full_pv_data = df[df['behavior_type'] == 'pv']

# 2. 分组聚合
pv_uv_puv = full_pv_data.groupby('dates').agg(
    pv=('user_id', 'count'),
    uv=('user_id', 'nunique')
)

# 3. 计算比例
pv_uv_puv['puv'] = (pv_uv_puv['pv'] / pv_uv_puv['uv']).round(1)

# 4. 重置索引（让 dates 变成一列，而不是索引）
pv_uv_puv = pv_uv_puv.reset_index()

# 5. 对应 SQL: delete from pv_uv_puv where dates is null;
pv_uv_puv.dropna(subset=['dates'], inplace=True)

# 查看最终结果
print(pv_uv_puv)
```

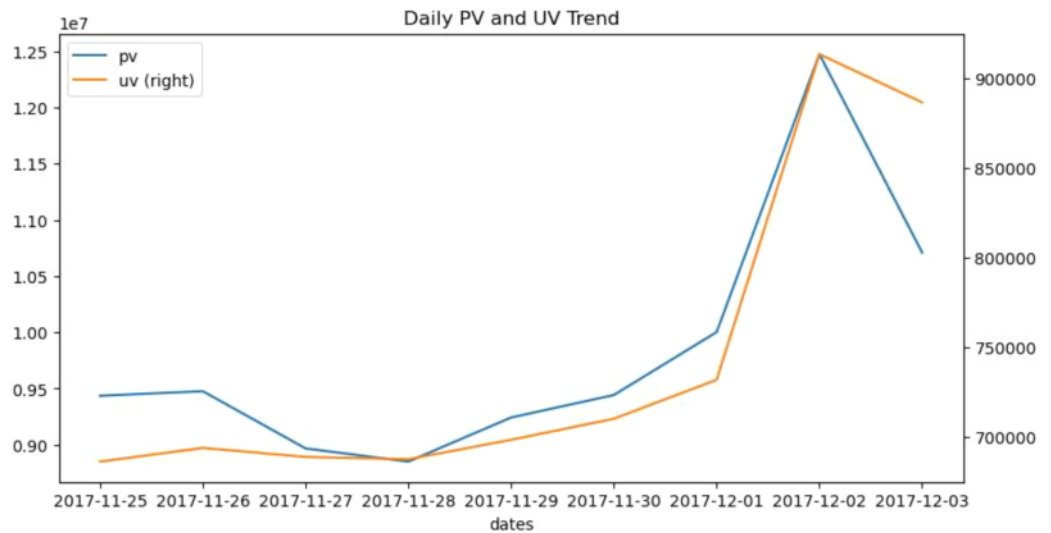
	dates	pv	uv	puv
0	2017-11-25	9435250	685907	13.8
1	2017-11-26	9475589	693559	13.7
2	2017-11-27	8966429	688495	13.0
3	2017-11-28	8849193	687244	12.9
4	2017-11-29	9241646	698074	13.2
5	2017-11-30	9442000	709828	13.3
6	2017-12-01	10002284	731566	13.7
7	2017-12-02	12475203	913314	13.7
8	2017-12-03	10709289	886400	12.1

In [17]:

```
import matplotlib.pyplot as plt

# 简单的趋势图
pv_uv_puv.plot(x='dates', y=['pv', 'uv'], secondary_y=['uv'], kind='line', figsi
```

```
plt.title('Daily PV and UV Trend')
plt.show()
```



留存情况

```
In [18]: user_active_days=df[['user_id','dates']].drop_duplicates()
```

```
In [19]: # 1. 关联数据以计算日期差 (类似于 SQL 的 a.user_id = b.user_id and a.dates <= b.dates)
retention_df = pd.merge(user_active_days, user_active_days, on='user_id', suffixes=('_a', '_b'))

# 2. 计算天数差
# 确保日期格式正确
retention_df['date_start'] = pd.to_datetime(retention_df['dates_a'])
retention_df['date_active'] = pd.to_datetime(retention_df['dates_b'])
retention_df['diff'] = (retention_df['date_active'] - retention_df['date_start']).dt.days

# 3. 过滤只保留 diff >= 0 的记录
retention_df = retention_df[retention_df['diff'] >= 0]

# 4. 计算留存数 (retention_0, retention_1, retention_3)
retention_counts = retention_df.groupby(['dates_start', 'diff']).size().unstack(fill_value=0)

# 5. 提取特定天数并计算留存率
# retention_1_rate = 次日留存数 / 当日总活跃数(diff=0)
retention_rate = pd.DataFrame(index=retention_counts.index)
retention_rate['retention_1'] = (retention_counts[1] / retention_counts[0]).round(4)

print(retention_rate.head())
```

dates_start	retention_1
2017-11-25	0.7863
2017-11-26	0.7767
2017-11-27	0.7838
2017-11-28	0.7913
2017-11-29	0.7955

```
In [20]: # 1. 计算跳失用户数 (仅有 1 条行为记录的用户)
# 对应 SQL: having count(behavior_type)=1
user_behavior_count = df.groupby('user_id')['behavior_type'].count()
```

```

bounce_user_count = (user_behavior_count == 1).sum()

# 2. 计算总 PV 数
# 对应 SQL: select sum(pv) from pv_uv_puv;
total_pv = df[df['behavior_type'] == 'pv'].shape[0]

# 3. 计算跳失率
bounce_rate = bounce_user_count / total_pv

print(f"跳失用户数: {bounce_user_count}")
print(f"总 PV: {total_pv}")
print(f"跳失率: {bounce_rate:.8f}")

```

跳失用户数: 92
 总 PV: 88596883
 跳失率: 0.0000104

```

In [21]: # 对应 SQL: create table retention_rate
retention_rate.reset_index().to_csv('retention_rate_result.csv', index=False)

```

时间序列分析

```

In [22]: # 对应 SQL: select dates, hours, count(if...) ... group by dates, hours
date_hour_behavior = df.pivot_table(
    index=['dates', 'hours'],
    columns='behavior_type',
    values='user_id',
    aggfunc='count',
    fill_value=0
)

# 对应 SQL 中的列顺序调整
# pivot_table 会根据 behavior_type 的字母顺序排列列名
# 我们手动调整为: pv, cart, fav, buy
cols = ['pv', 'cart', 'fav', 'buy']
# 确保数据中包含这些列 (防止某些行为在抽样数据中缺失)
existing_cols = [c for c in cols if c in date_hour_behavior.columns]
date_hour_behavior = date_hour_behavior[existing_cols]

# 重置索引, 让 dates 和 hours 变成普通的列
date_hour_behavior = date_hour_behavior.reset_index()

# 对应 SQL: order by dates, hours
date_hour_behavior = date_hour_behavior.sort_values(['dates', 'hours'])

print(date_hour_behavior.head())

```

behavior_type	dates	hours	pv	cart	fav	buy
0	2017-11-25	0	321406	19855	11114	6690
1	2017-11-25	1	378673	23207	12902	9456
2	2017-11-25	2	437162	26749	14870	12411
3	2017-11-25	3	432650	26286	14910	12379
4	2017-11-25	4	424641	24956	14275	11334

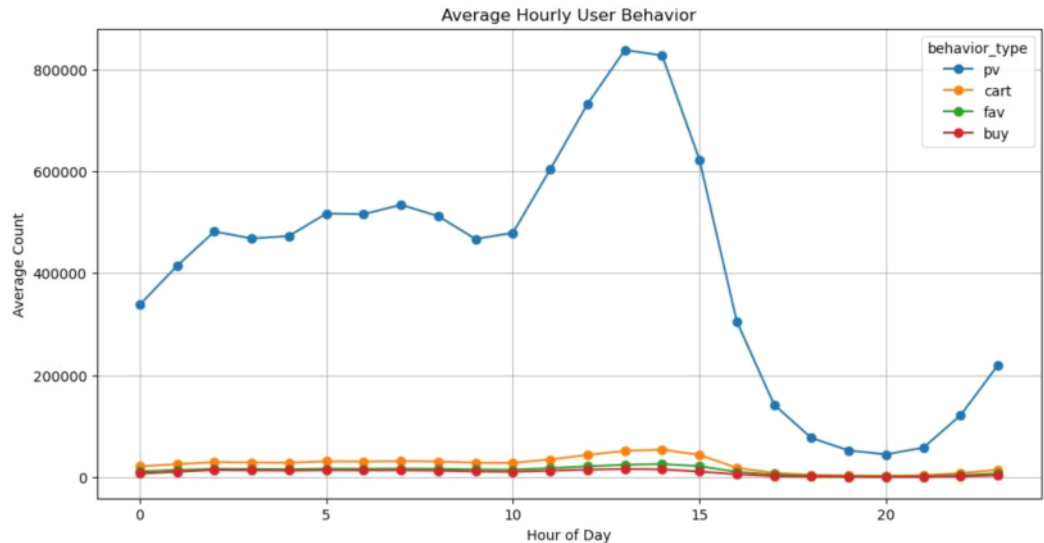
```

In [23]: import seaborn as sns

# 统计全天 24 小时的平均行为分布
hourly_avg = date_hour_behavior.groupby('hours')[['pv', 'cart', 'fav', 'buy']].m

```

```
# 绘图
hourly_avg.plot(kind='line', figsize=(12, 6), marker='o')
plt.title('Average Hourly User Behavior')
plt.xlabel('Hour of Day')
plt.ylabel('Average Count')
plt.grid(True)
plt.show()
```



用户转化率

```
In [24]: behavior_user_num = df.groupby('behavior_type')['user_id'].nunique().sort_index()
```

```
In [25]: # 2. 计算购买用户的比例
total_users = df['user_id'].nunique()
buy_users = behavior_user_num.get('buy', 0)
buy_user_ratio = buy_users / total_users

print("--- 各类行为去重用户数 ---")
print(behavior_user_num)
print(f"\n付费用户占比 (Buy Users / Total Users): {buy_user_ratio:.2%}")
```

--- 各类行为去重用户数 ---

```
behavior_type
pv      984080
fav     388261
cart    737393
buy     670370
Name: user_id, dtype: int64
```

付费用户占比 (Buy Users / Total Users): 67.85%

```
In [26]: # 1. 统计各类行为的总发生次数
# 对应 SQL: select behavior_type, count(*)
behavior_num = df['behavior_type'].value_counts().sort_index(ascending=False)

# 2. 计算各项转化指标
# 对应 SQL: select 2015807/89660670 (购买率)
pv_count = behavior_num.get('pv', 0)
buy_count = behavior_num.get('buy', 0)
fav_count = behavior_num.get('fav', 0)
```

```

cart_count = behavior_num.get('cart', 0)

buy_rate = buy_count / pv_count
fav_cart_rate = (fav_count + cart_count) / pv_count

print("\n--- 各类行为总量 ---")
print(behavior_num)
print(f"\n购买转化率 (Buy/PV): {buy_rate:.2%}")
print(f"收藏加购率 ((Fav+Cart)/PV): {fav_cart_rate:.2%}")

```

--- 各类行为总量 ---

```

behavior_type
pv      88596883
fav     2852536
cart    5466117
buy     1998944
Name: count, dtype: int64

```

购买转化率 (Buy/PV): 2.26%

收藏加购率 ((Fav+Cart)/PV): 9.39%

行为路径分析

```

In [27]: # 1. 以用户和商品分组，统计四种行为的发生次数
path_base = df.groupby(['user_id', 'item_id', 'behavior_type']).size().unstack(f

# 2. 确保四列齐全并标准化为 0 或 1 (对应 SQL 的 case when > 0)
for col in ['pv', 'fav', 'cart', 'buy']:
    if col not in path_base.columns:
        path_base[col] = 0
    path_base[col] = (path_base[col] > 0).astype(int)

# 重名列名，对应 SQL 的描述
path_base.columns = ['buy', 'cart', 'fav', 'pv'] # 注意顺序调整

```

```

In [28]: # 只统计最终购买了的行为 (where 购买了 > 0)
buy_paths = path_base[path_base['buy'] > 0].copy()

# 按照 SQL 的顺序: pv, fav, cart, buy
# 我们将 0/1 组合成 '1011' 这种格式
buy_paths['path_type'] = (
    buy_paths['pv'].astype(str) +
    buy_paths['fav'].astype(str) +
    buy_paths['cart'].astype(str) +
    buy_paths['buy'].astype(str)
)

# 统计每种路径的数量
path_count = buy_paths['path_type'].value_counts().reset_index()
path_count.columns = ['path_type', 'num']

```

```

In [29]: path_count

```


Out[29]:

	path_type	num
0	1001	941203
1	0001	513294
2	1011	252205
3	1101	86973
4	0011	75632
5	0101	25685
6	1111	13258
7	0111	1880

```
In [30]: # 定义映射字典
renhua_dict = {
    '0001': '直接购买了',
    '1001': '浏览后购买了',
    '0011': '加购后购买了',
    '1011': '浏览加购后购买了',
    '0101': '收藏后购买了',
    '1101': '浏览收藏后购买了',
    '0111': '收藏加购后购买了',
    '1111': '浏览收藏加购后购买了'
}

# 映射描述
path_count['description'] = path_count['path_type'].map(renhua_dict)

# 按照数量降序排列
path_result = path_count[['path_type', 'description', 'num']].sort_values('num',
path_result
```

Out[30]:

	path_type	description	num
0	1001	浏览后购买了	941203
1	0001	直接购买了	513294
2	1011	浏览加购后购买了	252205
3	1101	浏览收藏后购买了	86973
4	0011	加购后购买了	75632
5	0101	收藏后购买了	25685
6	1111	浏览收藏加购后购买了	13258
7	0111	收藏加购后购买了	1880

```
In [31]: # 1. 计算：购买行为中，没有收藏也没有加购的数量（直接购买或仅浏览购买）
# 对应 SQL: select sum(buy) from user_behavior_view where buy>0 and fav=0 and ca
no_fav_cart_buy = path_base[(path_base['buy'] > 0) & (path_base['fav'] == 0) & (

# 2. 计算：通过收藏或加购转化的购买数
# 对应 SQL: 2015807 - 1528016
```

```
total_buy_records = path_base[path_base['buy'] > 0].shape[0]
fav_cart_conversion_buy = total_buy_records - no_fav_cart_buy

# 3. 计算: (收藏+加购) 对 购买的贡献率
# 对应 SQL: 487791 / (2888255 + 5530446)
behavior_counts = df['behavior_type'].value_counts()
total_fav_cart = behavior_counts.get('fav', 0) + behavior_counts.get('cart', 0)

contribution_rate = fav_cart_conversion_buy / total_fav_cart

print(f"\n仅浏览/直接购买记录数: {no_fav_cart_buy}")
print(f"通过收藏加购转化的购买记录数: {fav_cart_conversion_buy}")
print(f"收藏加购对购买的转化效率: {contribution_rate:.2%}")
```

仅浏览/直接购买记录数: 1454497

通过收藏加购转化的购买记录数: 455633

收藏加购对购买的转化效率: 5.48%

RFM 模型

```
In [32]: # 1. 筛选购买行为
buy_data = df[df['behavior_type'] == 'buy']

# 2. 计算最近购买时间(R)和购买次数(F)
rfm_model = buy_data.groupby('user_id').agg(
    frequency=('user_id', 'count'),
    recent=('dates', 'max')
).reset_index()

rfm_model.head()
```

```
Out[32]:
```

	user_id	frequency	recent
0	2	7	2017-12-02
1	4	4	2017-11-30
2	11	1	2017-11-25
3	16	2	2017-12-01
4	17	1	2017-12-01

```
In [33]: # 评分函数: F分层
def get_f_score(f):
    if 100 <= f <= 262: return 5
    elif 50 <= f < 100: return 4
    elif 20 <= f < 50: return 3
    elif 5 <= f < 20: return 2
    else: return 1

# 评分函数: R分层
def get_r_score(r):
    # 假设 r 是字符串或 date 对象
    r = str(r)
    if r == '2017-12-03': return 5
    elif r in ['2017-12-01', '2017-12-02']: return 4
    elif r in ['2017-11-29', '2017-11-30']: return 3
    elif r in ['2017-11-27', '2017-11-28']: return 2
```

```

else: return 1

# 应用评分
rfm_model['fscore'] = rfm_model['frequency'].apply(get_f_score)
rfm_model['rscore'] = rfm_model['recent'].apply(get_r_score)

```

```

In [34]: # 1. 计算评分的平均值
f_avg = rfm_model['fscore'].mean()
r_avg = rfm_model['rscore'].mean()

# 2. 定义分类逻辑
def get_class(row):
    f = row['fscore']
    r = row['rscore']
    if f > f_avg and r > r_avg:
        return '价值用户'
    elif f > f_avg and r <= r_avg:
        return '保持用户'
    elif f <= f_avg and r > r_avg:
        return '发展用户'
    else:
        return '挽留用户'

# 3. 执行分类
rfm_model['class'] = rfm_model.apply(get_class, axis=1)

# 4. 统计各分区用户数
class_counts = rfm_model['class'].value_counts().reset_index()
class_counts.columns = ['class', 'user_num']

print(class_counts)

```

```

class  user_num
0  发展用户    284412
1  挽留用户    265403
2  价值用户    95583
3  保持用户    24972

```

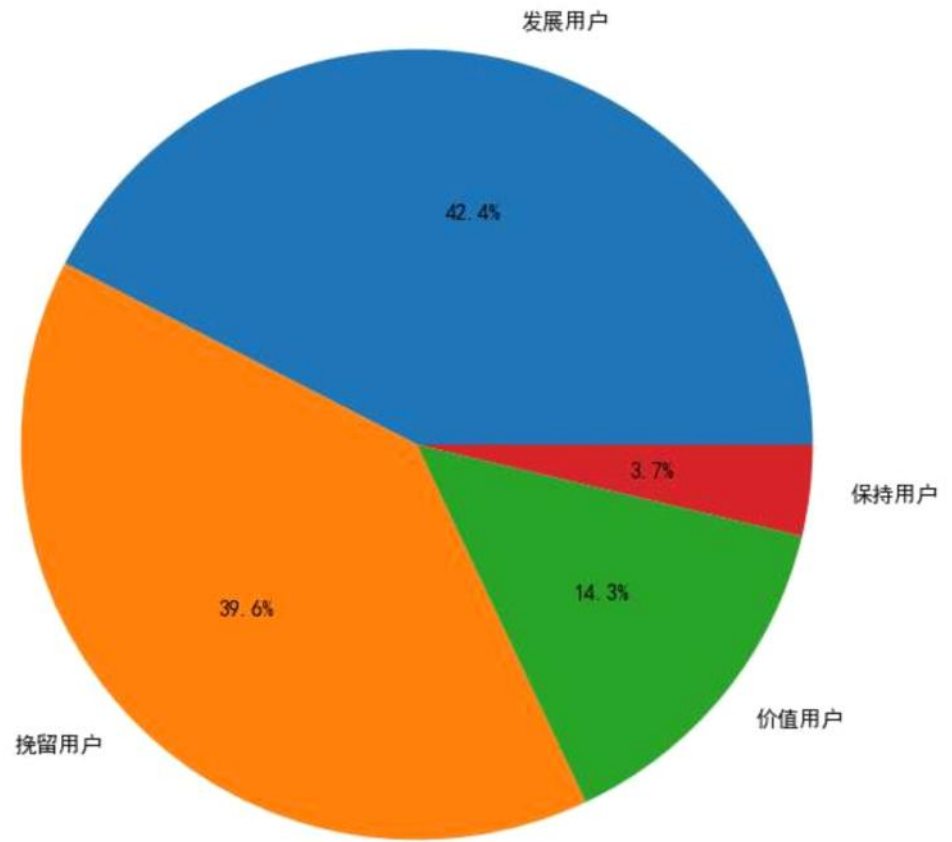
```

In [35]: # 解决中文显示问题
plt.rcParams['font.sans-serif'] = ['SimHei'] # 指定默认字体为黑体
plt.rcParams['axes.unicode_minus'] = False # 解决保存图像是负号 '-' 显示为方块的

# 然后再执行你的绘图代码
plt.figure(figsize=(8, 8))
plt.pie(class_counts['user_num'], labels=class_counts['class'], autopct='%1.1f%%')
plt.title('用户分层分布情况')
plt.show()

```

用户分层分布情况



商品按热度分类

```
In [36]: # 筛选浏览行为 (PV)
pv_df = df[df['behavior_type'] == 'pv']

# 1. 热门品类 Top 10
# 对应 SQL: select category_id, count(*) ... group by category_id
popular_categories = pv_df.groupby('category_id').size().nlargest(10).reset_index()

# 2. 热门商品 Top 10
# 对应 SQL: select item_id, count(*) ... group by item_id
popular_items = pv_df.groupby('item_id').size().nlargest(10).reset_index(name='pv')

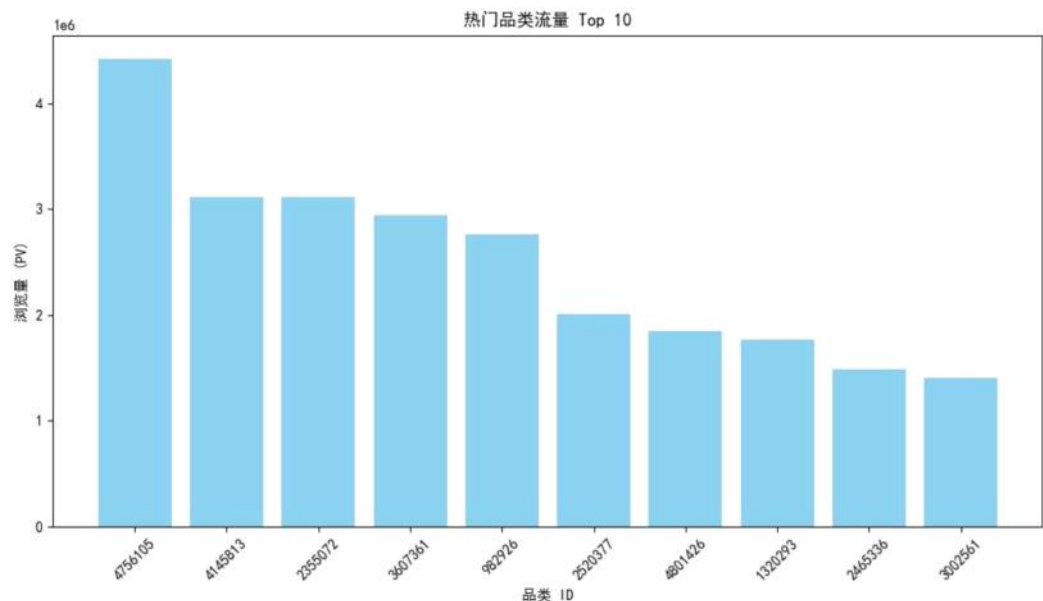
print("--- 热门品类 Top 10 ---")
print(popular_categories)
print("\n--- 热门商品 Top 10 ---")
print(popular_items)
```

```
--- 热门品类 Top 10 ---
category_id    pv
0      4756105  4426937
1      4145813  3115400
2      2355072  3110549
3      3607361  2941480
4       982926  2763686
5      2520377  2003073
6      4801426  1841977
7      1320293  1769460
8      2465336  1484268
9      3002561  1406307
```

```
--- 热门商品 Top 10 ---
item_id    pv
0    812879  29720
1   3845720  25290
2    138964  20927
3   2331370  19348
4   2032668  19075
5   1535294  17830
6     59883  17313
7   4211339  17235
8   3371523  17156
9   2338453  17044
```

```
In [37]: # 绘制热门品类的条形图 (使用方案一中解决中文的方法)
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

plt.figure(figsize=(12, 6))
plt.bar(popular_categories['category_id'].astype(str), popular_categories['pv'],
plt.title('热门品类流量 Top 10')
plt.xlabel('品类 ID')
plt.ylabel('浏览量 (PV)')
plt.xticks(rotation=45)
plt.show()
```



商品转化率

In []:

In []: